

IRONFORGE

Infraestrutura de produção e pendências de lançamento

Onde hospedar a API, o banco e os arquivos de mídia, com análise de plano gratuito e pago — e o levantamento completo do que falta para o produto ir ao ar.

Relatório técnico · 27 de agosto de 2026

Escopo: **ironforge** (app do aluno) · **ironforge-api** (backend) · **ironforge-web** (painel do personal)

1. Resumo para decisão

Este relatório responde duas perguntas: **onde subir a infraestrutura** e **o que ainda falta**. As duas respostas curtas estão aqui; o resto do documento é a fundamentação.

Infraestrutura: Fly.io na região `gru` (São Paulo) + Neon Postgres em `sa-east-1` + Cloudflare R2 para mídia.

Custo: **US\$ 10,77/mês (R\$ 58)** no lançamento · **US\$ 102,82/mês (R\$ 555)** com 1.000 alunos.

Sobre o gratuito: existe, economiza R\$ 44/mês, e **não vale a pena** — a seção 5 mostra o porquê com número.

Pendências: 35 itens, sendo **16 bloqueadores** de lançamento. O maior deles: o app não grava carga no servidor.

O ponto de partida é melhor do que costuma ser nesse estágio. Três aplicações funcionais, 313 testes passando, zero erro de tipo ou de lint. **O trabalho que falta é majoritariamente de ligação e operação**, não de construção.

2. Hospedagem da API

Comparação com preço confirmado nas páginas oficiais em 27 de agosto de 2026. Escala de referência: 50 pessoais e 1.000 alunos.

	Fly.io	Railway	Render	Hetzner	DigitalOcean
Instância pequena	US\$ 5,16 512 MB em <code>gru</code>	US\$ 5 + uso de RAM e vCPU	US\$ 7 (Starter)	€5,49 (menor é 4 GB)	US\$ 6
Em 1.000 alunos	US\$ 19	~US\$ 40–45	~US\$ 50–80	~US\$ 13	US\$ 12
Região São Paulo	SIM	não	não	não	não
Release command	sim	sim	só no pago	não — VPS puro	sim
Rollback	por linha de comando	botão	botão, o melhor	nenhum	botão

O cálculo que decide, e não é o preço. A camada de multi-tenancy (`withTenant`) custa **4 idas ao banco por requisição** — `BEGIN` , `set_config` , a consulta, `COMMIT` . Com o banco em São Paulo, qualquer plataforma sem região no Brasil paga esses quatro trajetos cruzando o hemisfério: **cerca de 130 ms × 4 ≈ 520 ms por requisição**. Isso não é preferência, é aritmética.

O Fly em `gru` custa **16 centavos de dólar a mais** por mês que o Railway e coloca API e banco no mesmo metrô. Por dezesseis centavos não há discussão.

Hetzner e DigitalOcean saem por dois motivos combinados, nenhum deles preço: não têm região no Brasil, e o Hetzner ainda exige que você construa deploy, TLS, health check, release command e rollback à mão. Para fundador solo, esse é o item mais caro do relatório — o seu tempo.

Alertas apurados: o free tier do Fly acabou para contas novas; a região `gru` tem sobretaxa de 61,5% sobre a Virgínia (já embutida nos US\$ 5,16); o Hetzner aplicou reajuste de até 3× em junho de 2026; o Render Free dorme após 15 minutos.

3. Banco de dados

	Neon	Supabase	Fly Postgres	Railway	AWS RDS
Mês 1	~US\$ 5	US\$ 25	US\$ 38	~US\$ 7	~US\$ 27
1.000 alunos	~US\$ 28	~US\$ 25–30	~US\$ 41	~US\$ 18	~US\$ 27
Região Brasil	sim	sim	sim	não confirmado	sim
Backup / PITR	7 dias por centavos	PITR custa US\$ 100/mês	incluso	~4 semanas	incluso, até 35 dias
Mensalidade fixa	nenhuma	US\$ 25	US\$ 38	por uso	por hora

O Supabase perde por aritmética, não por ideologia: o PITR dele custa US\$ 100/mês, quatro vezes a pilha inteira aqui recomendada. E o pacote que justificaria o plano Pro — Auth, Storage, Realtime — já está resolvido no código: autenticação própria com JWT e Argon2, storage próprio com SDK S3.

O Neon cortou preço depois da aquisição pela Databricks: storage caiu 80% e compute caiu de 15 a 25%. E o pooler dele hoje já suporta prepared statements, o que remove uma armadilha de configuração que existia.

Não use o plano gratuito do Neon. São 100 CU-h por mês; no menor passo de compute isso dá cerca de 400 horas, contra as 730 de um mês. Estourou no dia 20 e a mensagem é "*your compute is suspended until the next billing period*" — **o banco para de aceitar escrita e o aluno não consegue registrar a série.** Sem degradação gradual: é um interruptor. Grátis com penhasco custa mais que US\$ 5.

4. Fotos e vídeos — a pergunta do egress

É aqui que a escolha errada cobra caro, e a diferença não aparece no primeiro mês.

	Cloudflare R2	AWS S3 (SP)	Backblaze B2	Supabase
Storage US\$/GB	0,015	0,0405	0,00695	0,0213
Egress US\$/GB	ZERO, sempre	0,15	0,01	0,09
Free tier	10 GB/mês	100 GB de egress	10 GB	1 GB
CDN incluída	sim	não	via parceria	sim
Compatível com o código atual	sim, zero mudança	sim	sim	sim

A conta de egress com os números reais do projeto

Premissas do ADR 0007: 22 visualizações por aluno/mês, 70% do acervo em YouTube e 30% em upload próprio, clipe de 720p a 25 MB.

Escala	Armazenado	Egress/mês	R2	S3	B2
Mês 1 (~20 usuários)	0,45 GB	2,8 GB	US\$ 0	US\$ 0	US\$ 0
200 usuários	4,5 GB	28 GB	US\$ 0	US\$ 0	US\$ 0
1.000 alunos	7,5 GB	165 GB	US\$ 0	US\$ 10,05	US\$ 1,48
10.000 alunos	75 GB	1,65 TB	US\$ 0,98	US\$ 233	US\$ 15

O ponto de virada é 606 alunos ativos. É onde os 100 GB de egress gratuito do S3 acabam. Antes disso, R2 e S3 empatam em zero e escolher por preço é escolher por nada. Depois disso, o S3 passa a cobrar US\$ 0,15 por GB e o R2 continua cobrando zero — é a curva que leva a US\$ 233/mês com 10 mil alunos.

O Backblaze é duas vezes mais barato no storage e perde assim mesmo: precisa de CDN de terceiro para ter latência decente no Brasil, e o storage é a linha irrelevante da conta — US\$ 0,11 por mês em 7,5 GB. Otimizar pela metade uma linha de onze centavos não é economia, é distração.

Duas correções importantes na entrega de mídia

a) Aponte um domínio seu para o bucket desde o primeiro dia. Se URLs em

`*.r2.cloudflarestorage.com` vazarem para o app, para o banco ou para cache de cliente, trocar de provedor vira reescrita de URL. Com domínio próprio, virar as costas para o R2 é **uma mudança de DNS**.

b) URL assinada derrota a CDN. Cada assinatura é única na query string, então toda requisição é cache miss e bate na origem em vez do ponto de presença de São Paulo. Como o egress do R2 é grátis, isso não custa dinheiro — **custa latência, exatamente na academia com rede ruim**. A solução é gratuita e o modelo de dados já a permite: vídeo do catálogo da plataforma servido **público, sem assinatura**, para cachear na borda; vídeo do personal continua assinado.

Vídeo: arquivo cru ou serviço de streaming?

	MP4 cru do R2	Cloudflare Stream	Mux	bunny.net
Custo em 1.000 alunos	US\$ 0	US\$ 16,20	US\$ 12,00	~US\$ 5
HLS adaptativo	não	sim	sim	sim

Servir MP4 cru do R2 agora. Não contrate serviço de vídeo para lançar. A razão não é preço — US\$ 12 a 16 por mês não quebram ninguém. É que **70% do acervo é YouTube por decisão de arquitetura**, e o YouTube já faz bitrate adaptativo de graça. Você pagaria transcodificação para servir os 30% restantes de um acervo que no mês 1 tem 0,45 GB.

Mas o MP4 cru tem uma condição inegociável: `-movflags +faststart`. Um MP4 sem o átomo `moov` no início força o player a baixar o arquivo **inteiro** antes do primeiro quadro. Com faststart, o player começa a tocar em segundos. É uma flag, é grátis, e é a diferença entre "funciona" e "não funciona" no 3G de um subsolo de academia. **Verificar se o codificador do app já emite faststart — se não emitir, é bug de lançamento, não otimização.**

Gatilho para contratar streaming depois: quando a telemetria mostrar **P75 de tempo até o primeiro quadro acima de 3 segundos**, ou abandono de vídeo acima de 20%. Não quando a conta doer — ela não vai doer. Nessa hora, bunny.net por ~US\$ 5/mês. O contrato de API não muda: os campos `hlsUrl` e `durationSeconds` já estão reservados.

Fotos

Mesmo bucket, prefixos diferentes. O código já resolveu isso sem perceber: `STORAGE_BUCKET` é uma variável só, e as chaves já são hierárquicas (`coaches/{id}/videos/{id}.ext`). Avatar de personal, avatar de aluno e thumbnail de exercício entram no mesmo padrão sem tocar em código. Bucket separado exigiria um segundo cliente S3, um segundo conjunto de credenciais e um segundo caminho de configuração, sem um segundo caso concreto que justifique.

Renomear o bucket de `ironforge-videos` para `ironforge-media` antes do primeiro objeto entrar. É grátis agora e chato depois.

Redimensionamento: no cliente, na hora do upload. O app já tem `expo-image-picker` e `expo-image` ; o arquivo que sai do celular já é o final. Zero CPU no servidor, zero fornecedor novo, zero fila. Usar `sharp` no backend criaria um problema de infraestrutura — event loop bloqueado, risco de estouro de memória, binário nativo complicando a imagem Docker — para resolver um problema que não existe.

A regra que evita refazer isso em seis meses: guardar uma versão de cada imagem, no maior tamanho que algum dia será exibido (avatar 512 px, thumbnail 720 px). O `expo-image` reduz na tela de graça. Avatar de 512 px em JPEG dá ~40 KB; mil alunos são 40 MB, irrelevante. E no dia em que quiser seis tamanhos por breakpoint, o Cloudflare Images transforma direto sobre o R2 — o bucket não muda, as chaves não mudam, o banco não muda. É aditivo, não é migração.

5. Gratuito contra pago — onde o grátis quebra

Camada por camada, com o sintoma que o usuário final sente quando o plano gratuito atinge o limite. "Dá para validar" e "dá para operar" são coisas diferentes e estão separadas de propósito.

Camada	O grátis existe?	Onde quebra, na prática	Validar / operar	1º degrau pago
API	Não no Fly (acabou para conta nova). No Render, sim.	O Render Free dorme após 15 minutos e leva de 30 a 60 segundos para acordar . O aluno abre o app entre séries e olha um spinner por quase um minuto. Pior ainda: webhook que chega com a instância dormindo se perde em silêncio . E o Render não tem região no Brasil.	Render: nem validar . Fly com auto-stop: valida, não opera.	US\$ 5,16 Fly 512 MB 24/7
Banco	Neon Free: 100 CU-h, 0,5 GB, restore de 6 h.	Dois sintomas. O penhasco : 100 CU-h dão ~400 h num mês de 730; estourou, o banco para de aceitar escrita até virar o ciclo. O cold start empilhado : com a API também dormindo, o aluno paga o boot do container mais o retorno do banco.	Valida (demo, 5 pessoais). Não opera — nenhum produto com usuário real tem interruptor mensal.	~US\$ 5 Neon Launch, sem mensalidade fixa
Mídia	R2: 10 GB/mês, egress sempre grátis.	Praticamente não quebra nessa escala . 10 GB cobrem 1.300 alunos com o mix atual. Passando disso, o 11º GB custa um centésimo de dólar. Sem penhasco, sem surpresa. O que falta: backup — a durabilidade é altíssima, mas se o seu código apagar o objeto errado, ele foi.	Valida e opera . Única camada onde o grátis é resposta definitiva.	Não há degrau — é linear por GB.
CDN	Sim, e é o melhor negócio da lista: domínio próprio no R2 põe a rede da Cloudflare na frente sem custo, com presença em São Paulo.	Não quebra por preço — quebra por URL assinada, como explicado na seção anterior.	Opera indefinidamente .	—
E-mail	Resend: 3.000/mês, com teto de 100 por dia .	O teto diário, não o mensal . Convidar 50 alunos de uma vez somado aos resets do dia já estoura. E sem domínio verificado o Resend só envia para o e-mail da própria conta — inútil em produção.	Valida com folga. Opera até ~30 pessoais ativos.	US\$ 20 (Resend Pro) ou US\$ 0,10/mil (AWS SES)

Monitoramento	Sentry: 5.000 eventos/mês. Better Stack: 10 monitores.	No Sentry, erros e transações dividem a mesma cota — ligar tracing sem amostragem queima os 5 mil numa tarde. Configurar <code>tracesSampleRate: 0.1</code> desde o começo. No Better Stack, log com retenção de 3 dias: o bug de sexta é investigado na terça e o log sumiu.	Opera bem nessa escala.	US\$ 26 (Sentry Team)
Painel	Vercel Hobby: US\$ 0.	A política é literal: " <i>restricted to non-commercial personal use only</i> ". No dia em que o primeiro personal pagar, vira uso comercial e o Hobby deixa de ser opção.	Opera enquanto o beta for gratuito.	US\$ 9,20 2ª máquina no Fly, metade do Vercel Pro

Não use o UptimeRobot. Os termos de serviço proíbem uso comercial desde dezembro de 2024. É a primeira sugestão que aparece em qualquer busca por "monitoramento grátis" e não serve para um produto que vai cobrar.

6. Os dois cenários, fechados

Cenário mínimo — o máximo por perto de zero

Item	Escolha	US\$/mês
API	Fly 512 MB com desligamento automático	~1,90
Banco	Neon Free	0,00
Mídia e CDN	R2 free tier	0,00
E-mail	Resend Free	0,00
Monitoramento	Sentry Free + Better Stack Free	0,00
Painel	Vercel Hobby	0,00
Domínio <code>.com.br</code>	registro.br, R\$ 40/ano	0,62
Infraestrutura		US\$ 2,52 · R\$ 14
Apple Developer	US\$ 99/ano, amortizado	8,25
Total real	Google Play cobra US\$ 25 uma única vez	US\$ 10,77 · R\$ 58

Não é zero. São R\$ 14/mês de infraestrutura e **R\$ 58/mês** contando a Apple — que é obrigatória e que ninguém lembra de somar.

O que se perde: cold start empilhado de 2 a 4 segundos toda primeira abertura do dia; o penhasco de CU-h que trava a escrita no meio do mês; restore de apenas 6 horas, ou seja, dado apagado ontem está perdido; webhook perdido em silêncio; e o Vercel Hobby proibindo uso comercial no dia do primeiro pagamento.

Risco aceitável apenas para demonstração e para os cinco primeiros pessoais que você conhece pessoalmente. Deixa de ser aceitável no dia em que um desconhecido instalar o app.

Cenário custo-benefício — a recomendação de verdade

Item	Escolha	US\$/mês
API	Fly 512 MB em gru , 24 horas por dia	5,16
Egress da API	~1 GB	0,04
TLS	10 certificados grátis por organização	0,00
Banco	Neon Launch com PITR de 7 dias	4,95
Mídia e CDN	R2, dentro do free tier	0,00
E-mail	Resend Free com domínio verificado	0,00
Monitoramento	Sentry + Better Stack, ambos free	0,00
Painel	Vercel Hobby enquanto o beta for gratuito	0,00
Domínio	—	0,62
Infraestrutura		US\$ 10,77 · R\$ 58
Apple Developer	amortizado	8,25
Total		US\$ 19,02 · R\$ 103

A diferença entre os dois cenários é US\$ 8,25/mês — R\$ 44,55.

Esses R\$ 44,55 compram: fim do cold start, fim do penhasco de CU-h, recuperação de 7 dias em vez de 6 horas, e nenhum webhook perdido.

Recomendação: pule o cenário gratuito. Uma hora sua depurando "por que o app demorou 4 segundos para o aluno da academia X" custa mais que R\$ 44.

Custo na escala de 1.000 alunos

Camada	Mês 1 (US\$)	1.000 alunos (US\$)
API + egress + TLS	5,20	19,00
Postgres	4,95	28,00
Mídia, CDN e vídeo	0,00	0,00
E-mail	0,00	20,00
Monitoramento	0,00	26,00
Painel	0,00	9,20
Domínio	0,62	0,62
Total	US\$ 10,77 · R\$ 58	US\$ 102,82 · R\$ 555

Nenhuma linha tem cauda longa. Não há item que possa explodir dez vezes sem que o número de usuários exploda junto — porque a linha que normalmente explode, egress de vídeo, é **zero por construção** no R2.

A linha do Postgres em escala é estimativa a partir dos preços unitários oficiais, não de plano publicado. Faixa realista: US\$ 15 a 35/mês. É a única linha com incerteza material na tabela.

Com quantos alunos pagantes isso se paga? Supondo R\$ 29,90 por aluno ou R\$ 149 por personal — premissa, ajuste ao seu modelo:

Mês 1 (R\$ 103): 4 alunos pagantes, ou 1 personal.

Escala de 1.000 alunos (R\$ 555): 19 alunos dos mil, ou 4 pessoais dos cinquenta. A infraestrutura consome **1,9% da receita bruta**.

Do mês 1 à escala de mil alunos, a conta cresce 9,5 vezes enquanto a base cresce 50. **A infraestrutura nunca é o problema deste negócio.**

7. Lock-in e ordem de execução

7.1 O que trava e o que é trocável

Uma pilha barata que prende é cara. Esta não prende — com uma exceção.

A única trava real: `EXPO_PUBLIC_API_URL` compilada no binário. Variáveis com esse prefixo são embutidas no bundle em tempo de build. Publicar com `https://ironforge.fly.dev` amarra o projeto ao Fly pelo tempo que os usuários levarem para atualizar — o que, em app de academia, é indefinido.

Boa notícia apurada: o resgate existe e é rápido. O app já tem canal de atualização OTA configurado com política de versão por `appVersion` — um `eas update` reconstrói o bundle com a URL nova e entrega sem passar pela loja. **Minutos, não meses.** Ainda assim: domínio próprio no dia 1, porque não há motivo para depender de plano de resgate.

Peça	Trabalho para trocar de fornecedor	Bloqueia?
API	O mesmo <code>Dockerfile</code> roda em ECS, Cloud Run, Railway ou Render. Troca-se o arquivo de configuração da plataforma.	não
Postgres	<code>pg_dump</code> e <code>pg_restore</code> . Schema, políticas de RLS e migrations são Postgres puro.	não
Mídia	Sincronizar os objetos e trocar 5 variáveis de ambiente. Zero código — o cliente S3 já é genérico.	não
Vídeo para HLS	Preencher <code>hlsUrl</code> e <code>durationSeconds</code> , campos já reservados. Aditivo, nenhum cliente muda.	não
E-mail	Trocar um adaptador.	não
Painel	<code>next build</code> roda em qualquer lugar.	não

Três proibições que preservam essa liberdade: não adotar o driver serverless proprietário do Neon (trocaria uma migração de `pg_dump` por uma reescrita da camada de dados); não adotar Auth ou Storage do Supabase (a autenticação própria já existe e é melhor); e não usar DNS interno da plataforma na `DATABASE_URL` nem ler variáveis específicas do provedor dentro de regra de negócio.

7.2 Ordem de execução

Bloqueiam o primeiro deploy

1. **Registrar o domínio e apontar os subdomínios da API e do CDN.** É o passo 1 porque é a única decisão irreversível da lista. Nada vai para a loja antes disso.
2. **Corrigir o RLS** — as 12 tabelas têm `ENABLE ROW LEVEL SECURITY` e nenhuma tem `FORCE`, e a role `ironforge_app` é `NOLOGIN`. Detalhado na seção 8.2. Sem isso o isolamento entre pessoais é decoração: os testes passam e o vazamento existe.
3. **Criar o arquivo de configuração do Fly** com região `gru`, health check em `/health` e comando de release `node dist/shared/db/migrate.js`.

- 4. **Provisionar o Neon** em São Paulo, com PITR de 7 dias ligado.
- 5. **Provisionar o R2**, bucket `ironforge-media` — não `ironforge-videos`, renomeie antes do primeiro objeto — com versionamento e expiração de versões antigas em 30 dias. É a rede contra apagamento acidental.
- 6. **Subir**, conferir o health check e rodar os seeds contra produção.

Erro encontrado na documentação: a ADR 0008 manda usar `npm run db:migrate` como comando de release. **Isso falharia** — esse script usa `tsx`, que não existe na imagem final de produção. O script correto já existe no repositório (`db:migrate:prod`), e o `docs/deploy.md` já está certo. **Corrigir a ADR**, senão o primeiro deploy quebra no passo de migration.

Semana 1, antes do primeiro personal externo

- 7. **Verificar o domínio no provedor de e-mail**, com DKIM e SPF. Sem isso o convite cai no spam do Gmail e o personal conclui que o produto está quebrado.
- 8. **Sentry nos três projetos**, com amostragem de tracing em 10% para não queimar a cota.
- 9. **Monitor externo** no health check a cada 3 minutos, com alerta por e-mail. Não confie no alerta nativo da plataforma — não foi possível confirmar que funcione no plano base de nenhuma das três.
- 10. **Dump semanal do banco para fora do provedor.** É um agendamento de uma linha e é a diferença entre um susto e o fim do projeto.
- 11. **Aceitar formalmente os contratos de tratamento de dados** dos fornecedores e declarar a transferência internacional na política de privacidade. O prazo da Resolução ANPD nº 19/2024 **venceu em agosto de 2025** — isso é dívida vencida, não planejamento.

Antes do primeiro vídeo próprio subir

- 12. **Verificar `+faststart`** no codificador do app.
- 13. **Separar vídeo público de vídeo privado** na entrega, para o catálogo da plataforma cachear na borda em São Paulo.

Só quando o gatilho disparar

Gatilho	Ação	Custo adicional
Primeiro personal pagante	Sair do Vercel Hobby, painel para o Fly	+US\$ 9,20
P75 de tempo até o primeiro quadro acima de 3 s	Contratar streaming	+~US\$ 5
Acima de 3.000 e-mails/mês, ou teto diário mordendo	Plano pago de e-mail, ou migrar para SES	+US\$ 0 a 20
Existir receita recorrente	Segunda instância da API	+US\$ 9,20
Acima de 600 alunos ativos	Nada — o R2 já está certo. Só conferir que ninguém migrou para S3 por engano	US\$ 0
Acima de 2.000 alunos, ou contrato com RPO definido	Abrir a decisão de arquitetura seguinte, com uso real medido	—

O que foi deliberadamente não recomendado: Kubernetes, microserviços, multi-região, infraestrutura como código, Redis, WAF, pooler externo, réplica de leitura e transcodificação. Nenhum tem um segundo caso

concreto antes de mil alunos.

8. Segurança e LGPD — o que a infraestrutura precisa atender

Revisão de código nos três repositórios, não pentest. Todo achado abaixo foi confirmado abrindo o arquivo citado. Dois deles eu verifiquei uma segunda vez, por conta própria, antes de escrever aqui — estão marcados.

8.1 O que já está bem feito

Vale registrar, porque muda a prioridade do resto: a disciplina de segurança do backend está acima da média para este estágio.

- Senha com **Argon2id** nos parâmetros recomendados pela OWASP (`memoryCost: 19456, timeCost: 2`).
- **Refresh token rotativo com detecção real de reuso** e revogação de família inteira — não é só descrito na documentação, o código faz (`modules/auth/service.ts:136-165`). Tokens gravados como hash SHA-256, nunca em texto puro no banco.
- Recurso de outro tenant responde **404, nunca 403** — decisão deliberada contra enumeração de identificadores, documentada no próprio OpenAPI e aplicada de fato nos serviços.
- Escopo de dado sempre derivado do JWT, nunca de parâmetro de URL cru.
- Vídeo servido por **URL assinada com validade de 1 hora**; upload vai direto ao bucket, o binário nunca passa pela API.
- Variáveis de ambiente validadas com Zod no boot — falta variável obrigatória, o servidor não sobe.
- **Zero segredo no histórico do git** dos três repositórios. Verificado com `git log --all -- .env` .

8.2 O achado mais grave: o RLS provavelmente não está protegendo nada

Verificado por mim, além do agente. Confirmado.

As migrations criam políticas de Row Level Security e uma role `ironforge_app` para usá-las. A intenção é defesa em profundidade e está certa — o comentário em `0002_training_qls.sql` diz literalmente que a checagem de aplicação continua sendo o mecanismo primário e que as políticas existem para pegar um bug nela.

O problema é que **a segunda camada não está armada**:

- A role `ironforge_app` é criada como `NOLOGIN` e **em nenhum lugar do código a conexão assume essa role** — não existe `SET ROLE` nem `SET SESSION AUTHORIZATION` em todo o repositório.
- A aplicação conecta com o usuário cru da `DATABASE_URL` , que é o mesmo usuário que roda as migrations — ou seja, o **dono das tabelas**.
- Em Postgres, **o dono da tabela ignora RLS por padrão**, a menos que exista `FORCE ROW LEVEL SECURITY` . Não existe: zero ocorrências no repositório.

Consequência prática: se qualquer serviço novo esquecer o filtro por `coachId` ou `athleteId` , o banco não vai barrar. E isso quase aconteceu — o comentário da própria migration `0002` menciona ter corrigido condições que deixavam um tenant ler e escrever perfil de aluno de outro. A rede de proteção existe no papel e está desligada na prática.

Correção, antes de subir com dado real:

1. Separar os dois usuários: um roda as migrations e é dono das tabelas; a `DATABASE_URL` da aplicação aponta para um usuário de runtime distinto, que herda `ironforge_app`.
2. Adicionar `ALTER TABLE ... FORCE ROW LEVEL SECURITY` em todas as tabelas com política — protege até no caso de alguém rodar a aplicação com superusuário por engano.
3. Escrever um teste de integração que conecta como o usuário de runtime e tenta ler linha de outro tenant sem passar pela camada de aplicação. Esse teste não existe hoje, e é ele que impede a regressão voltar.

8.3 Onde o token fica guardado

App do aluno — token em texto puro, e refresh token jogado fora

Verificado por mim, além do agente. Confirmado.

Dois problemas no mesmo lugar, `features/auth/store.ts`:

- O access token é persistido em **AsyncStorage, que não é criptografado** e não tem controle de acesso do sistema operacional. Num aparelho com root, jailbreak ou backup extraído, o token é legível. A diretriz correta é `expo-secure-store`, que usa Keychain no iOS e Keystore no Android.
- O schema da resposta de login declara `refreshToken` (`auth/api.ts:17`), mas a linha 115 do store faz `const { accessToken, user } = response` — **o refresh token é descartado e nunca chega a ser guardado**. Na prática, o app não implementa renovação de sessão: quando o access token expira em 15 minutos, a sessão simplesmente cai e o aluno precisa logar de novo.

Esse segundo ponto merece atenção especial porque **não aparece como bug** — aparece como "o app desloga sozinho de vez em quando". O backend já tem toda a mecânica de rotação de refresh token pronta e bem feita; ela só nunca foi conectada no cliente. É o P0 nº 12 do PRD ("sessão persistente, sacrificar logins") não implementado.

Painel do personal — refresh token de 30 dias em localStorage

`shared/lib/auth/session.ts` grava access token e refresh token em `localStorage`. Não há cookie `httpOnly`, não há CSP configurado no `next.config.ts`. Qualquer XSS no painel — inclusive vindo de dependência de terceiro — leva embora um token de 30 dias que dá acesso aos dados de saúde de todos os alunos daquele personal.

A correção completa (refresh token em cookie `httpOnly` + `Secure` + `SameSite`) é decisão de arquitetura de sessão, não conserto de uma linha. Mitigação parcial imediata: reduzir o TTL do refresh token e adicionar CSP básico.

8.4 Segredos e configuração

Segredo	Ação obrigatória antes do go-live
<code>JWT_SECRET</code>	Gerar valor aleatório novo, mínimo 32 caracteres, diferente por ambiente . Nunca reaproveitar o de desenvolvimento.
<code>DATABASE_URL</code>	Credencial nova, com usuário dedicado de runtime — ver a correção de RLS acima, os dois assuntos são o mesmo.

`STORAGE_ACCESS_KEY` /
`SECRET`

Par de chaves novo, **escopado só ao bucket**, nunca a chave-mestra da conta do provedor.

Dois ajustes menores encontrados: o `.gitignore` do painel cobre `.env` e `.env*.local`, mas **não cobre** `.env.production` — buraco pronto para alguém cair nele num `git add .` futuro. E nenhuma variável `EXPO_PUBLIC_*` existe hoje no app, o que é a disciplina certa: tudo que tem esse prefixo é público por definição e nunca deve receber chave de terceiro.

8.5 Bucket de mídia

O código está correto: URL assinada com validade, upload direto, nenhuma ACL pública em lugar nenhum. **O risco não está no código, está na configuração do bucket.** Se o bucket for público por configuração de infraestrutura, a assinatura vira decoração — qualquer um monta a URL direta e acessa sem token, para sempre.

Ação não verificável por código: no console do provedor, confirmar `Block all public access` ligado no bucket de produção. É a única forma real de uma URL de vídeo de aluno vazar permanentemente.

8.6 LGPD — o que a lei exige e o que é boa prática

Separando as duas coisas, porque misturá-las gera trabalho desnecessário e deixa passar o que importa.

Exigência legal

- **Base legal para dado sensível.** Peso corporal, lesão e restrição física são dado de saúde (art. 5º, II) e exigem base do art. 11 — na prática, **consentimento específico e destacado**, coletado separado do aceite genérico de termos. Não pode ser uma caixinha só.
- **Cuidado com a base "tutela da saúde".** Ela existe no art. 11, mas pressupõe profissional de saúde ou entidade sanitária. Educador físico não se enquadra nisso no sentido estrito da lei. Não usar essa base sem parecer jurídico.
- **Exclusão de conta e dados** (art. 18, IV e VI). Precisa de fluxo real dentro do produto — hoje **não existe rota de auto-exclusão na API**. É também exigência da Apple, então o item aparece duas vezes por motivos diferentes.
- **Encarregado de dados e canal de contato visíveis** (art. 41). Para operação pequena pode ser pessoa física designada, mas o canal precisa existir e estar publicado.
- **Quem mais vê o dado.** Precisa estar explícito que o personal enxerga peso, lesão declarada e vídeo do aluno. Para o aluno é óbvio que o personal vê o treino; não é óbvio que vê o resto.

Boa prática, não exigência

- **A LGPD não exige hospedagem no Brasil.** O art. 33 permite transferência internacional com cláusulas contratuais padrão ou consentimento específico. Hospedar fora é legal, desde que a política informe a transferência e a hipótese usada.
- Ainda assim, região São Paulo é preferível — por latência, e porque **simplifica a conversa comercial**: o personal vai perguntar onde ficam os dados dos alunos dele, e "no Brasil" é uma resposta que não precisa de explicação.

9. O que já está pronto

Antes da lista do que falta, o que já existe — porque muda a leitura de tudo. O IronForge não é um protótipo: é um sistema com três aplicações funcionais e suíte de testes verde.

Repositório	Testes	O que já funciona
<code>ironforge-api</code> Fastify 5 + Drizzle + Postgres 16	112 passando 86,1% cobertura	34 rotas cobrindo auth com refresh token rotativo, catálogo de exercícios, templates de plano, atribuição individual e em massa, sessões e séries de treino, histórico de carga, recordes pessoais e biblioteca de vídeo. Helmet, CORS, rate limit e JWT configurados. RLS multi-tenant nas migrations. OpenAPI 3.1 gerado do próprio servidor. Dockerfile multi-stage pronto.
<code>ironforge-web</code> Next.js 16 + React 19	85 passando	Painel do personal com dashboard, tabela de alunos com filtros, convite e remoção, builder de plano com drag-and-drop, atribuição em massa, biblioteca de vídeo com fila de upload, perfil do personal e página pública por slug.
<code>ironforge</code> Expo + React Native	116 passando	App do aluno com autenticação, onboarding, home, lista de treinos, logger de treino com teclado numérico e timer de descanso, tela de exercício com vídeo de demonstração, resumo de sessão, progresso e histórico.

Leitura honesta: o volume de trabalho restante é menor do que parece pela lista abaixo. A maior parte do que falta é *ligação e operação* — conectar peças que já existem e colocar o conjunto num servidor — e não construção de funcionalidade nova.

10. O que ainda falta — relatório completo

30 itens, organizados em cinco marcos de execução. Todos estão registrados no Linear, no projeto Ironforge, com descrição, escopo, critério de aceite e dependências ligadas. O identificador entre parênteses é o card.

BLOQUEADOR impede o produto de ir ao ar. **ALTA** deveria estar pronto no lançamento. **MÉDIA** e **BAIXA** podem vir depois.

Marco 1 — Ambiente de produção no ar

Nada de produto novo aqui: só tirar o sistema do localhost.

Card	Item	Peso	Por que importa
SPA-94	Postgres gerenciado + migrations	BLOQUEADOR	Hoje o banco só existe no <code>docker-compose</code> local. As migrations precisam rodar como passo de release, nunca no boot — o migrator do Drizzle não usa advisory lock e duas instâncias subindo juntas colidem. Inclui o seed do catálogo de exercícios, sem o qual o app abre vazio.
SPA-95	Deploy da API	BLOQUEADOR	O <code>Dockerfile</code> está pronto. Falta o ambiente e as variáveis: <code>JWT_SECRET</code> novo, <code>CORS_ORIGINS</code> com a origem real do painel, <code>NODE_ENV=production</code> . Atenção: <code>CORS_ORIGINS</code> vazio desliga o CORS e o painel para de funcionar sem erro óbvio.
SPA-96	Deploy do painel com MSW desligado	BLOQUEADOR	O painel ainda carrega o <code>MswGate</code> no layout raiz. Build de produção não deve levar mock nenhum, e isso precisa ser verificado no ambiente real, não presumido.
SPA-97	Object storage + CDN	ALTA	O código de presigned upload já existe e aponta para um MinIO local. Com as variáveis vazias o upload de arquivo desliga e só o YouTube funciona — é o plano B aceitável se o prazo apertar.
SPA-98	Domínio, DNS e TLS	ALTA	O app de produção exige host estável: <code>EXPO_PUBLIC_API_URL</code> é obrigatória no build de release. E universal links do convite só funcionam com domínio.
SPA-99	CI nos três repositórios	ALTA	Nenhum repo tem <code>.github/</code> . 313 testes existem e nada impede um merge quebrado. Inclui checagem de que o OpenAPI não ficou dessincronizado.
SPA-100	Backup do banco com restore testado	ALTA	O ativo do produto é o histórico de carga do aluno. Backup que nunca foi restaurado não é backup, é esperança.
SPA-101	Sentry e alerta de 5xx	MÉDIA	Sem rastreamento de erro, um crash no app do aluno é invisível — a descoberta vem pela avaliação de uma estrela na loja.

Marco 2 — App consumindo a API real

Meta: *personal convida, aluno aceita, recebe plano, registra carga, e a carga persiste.*

Card	Item	Peso	Por que importa
SPA-102	Persistir sessão e séries na API	BLOQUEADOR	O achado mais grave da análise. É o P0 nº 4 do PRD — "registro de carga persistente, sua dor #1" — e não existe de verdade. A API expõe <code>POST /sessions</code> , <code>/sessions/{id}/sets</code> e <code>GET /load-history</code> , e o app não chama nenhuma dessas rotas: <code>features/workout/store.ts</code> guarda tudo só no dispositivo. Trocar de celular ou reinstalar zera o histórico — exatamente o que o produto promete impedir.
SPA-103	Progresso e histórico reais	BLOQUEADOR	As telas de progresso e histórico leem de arquivos de mock. O aluno vê gráfico de outra pessoa.
SPA-104	Convite por deep link real	BLOQUEADOR	<code>entities/invite/lib/decode-token.ts</code> valida o token contra um <code>MOCK_TOKENS</code> fixo no código. É a porta de entrada de todo aluno do produto, e ela é falsa.
SPA-108	Remover login de dev do build	BLOQUEADOR	<code>dev-accounts.ts</code> e o seletor de login permitem entrar em contas de teste com um toque. Num build de loja, isso é acesso a conta sem senha na mão de qualquer pessoa.
SPA-107	Remover a área do personal do app	BLOQUEADOR	Decisão tomada: o personal usa só o painel web. As quatro telas de <code>app/(coach)/</code> leem de mock, e tela com dado falso é motivo de rejeição na revisão da App Store. Inclui tratar o personal que tentar entrar pelo app, para não cair em loop de redirect.
SPA-121	Remover a tela de notificações mockada	ALTA	Mesma situação: caixa de entrada que nunca recebe nada, sem push por trás. Volta junto com o push de verdade.
SPA-105	Home, treinos e perfil sem mock	ALTA	Três telas principais ainda montadas sobre dados fictícios.
SPA-106	Fila offline do registro de carga	ALTA	Subsolo de academia é o pior lugar de rede que existe, e é exatamente onde o app é usado. Sem fila local, série registrada durante queda de sinal some.

Marco 3 — Ciclo de vida de conta

O que um usuário externo precisa para se virar sozinho, sem te ligar.

Card	Item	Peso	Por que importa
SPA-109	Recuperação de senha	BLOQUEADOR	A API não tem rota de reset — só register, login, refresh, logout e me. Todo usuário que esquecer a senha vira um chamado manual para você.

SPA-110	E-mail transacional	BLOQUEADOR	Não existe provedor de e-mail em nenhum dos três repos. Sem ele não há reset de senha nem entrega automática de convite. Exige domínio remetente com SPF e DKIM, senão cai em spam e o aluno nunca entra.
SPA-111	Endurecer as rotas de auth	ALTA	O rate limit é o mesmo (200/min) para listar exercício e para tentar senha. Falta limite específico e bloqueio progressivo por conta.

Marco 4 — Publicação nas lojas

Card	Item	Peso	Por que importa
SPA-114	Contas Apple e Google	BLOQUEADOR	Comece por aqui, hoje. É o único item com espera de terceiro: aprovação da Apple leva dias, e conta pessoal nova no Google exige 12 testadores por 14 dias antes de liberar produção. Começar tarde atrasa o lançamento mais que qualquer código.
SPA-113	Privacidade, termos e LGPD	BLOQUEADOR	Sem URL pública de política de privacidade a submissão nem é aceita. E o produto trata dado de saúde — lesões, restrições, peso — o que puxa obrigação real, incluindo exclusão de conta pelo próprio app.
SPA-112	Perfil de produção no EAS	BLOQUEADOR	<code>eas.json</code> tem apenas o perfil <code>preview</code> . Não há perfil de produção, não há bloco de submit, e a versão está em 0.1.0. Hoje não existe binário submissível.
SPA-116	Smoke test em produção	ALTA	313 testes cobrem as peças isoladas. Ninguém testou as três juntas contra o ambiente real, e o fluxo do primeiro dia atravessa os três repositórios.
SPA-115	Ficha de loja	ALTA	Screenshots nas resoluções exigidas, descrição, classificação etária e os formulários App Privacy e Data Safety — que precisam bater com a política publicada.
SPA-117	Licença no repo do app	BAIXA	Os outros dois repositórios têm <code>LICENSE</code> ; o do app não.

Marco 5 — Primeira coisa depois do lançamento

Card	Item	Peso	Por que importa
SPA-122	Pesquisa de cobrança	ALTA	Precisa fechar antes de escrever qualquer linha de cobrança: modelo, gateway, regra das lojas e parte fiscal. Já decidido: assinatura vendida só pelo painel web, nunca dentro do app, para ficar fora da comissão da Apple e do Google.
SPA-118	Cobrança recorrente	MÉDIA	O esqueleto já existe no banco: limite de alunos por plano e o erro de limite atingido. Dá para lançar cobrando por fora, no Pix manual, enquanto isso é construído.

SPA-119	Push notification	MÉDIA	Aderência ao treino é o que segura o aluno. Lembrete é a alavanca mais barata que existe para isso.
SPA-120	Métricas de ativação e retenção	MÉDIA	Sem métrica, o roadmap seguinte vira opinião. A pergunta a responder é: de cada 10 convites, quantos alunos treinam na primeira semana?

Achados da varredura técnica — o que não estava na lista inicial

Uma varredura independente dos três repositórios encontrou oito itens que a leitura anterior não tinha pego. Quatro deles são graves. Os dois primeiros eu verifiquei pessoalmente, abrindo os arquivos, antes de registrar.

Card	Item	Peso	Achado
SPA-123	RLS não está protegendo nada	BLOQUEADOR	Detalhado na seção 8.2. A aplicação conecta como dona das tabelas e dono de tabela ignora RLS. A rede de proteção existe no papel e está desligada.
SPA-124	App descarta o refresh token	BLOQUEADOR	Detalhado na seção 8.3. O app não renova sessão — desloga o aluno a cada 15 minutos — e guarda o token em armazenamento não criptografado.
SPA-126	Personal não vê nem troca o plano do aluno	BLOQUEADOR	O drawer de detalhe do aluno não tem uma única menção a plano ou atribuição . Do outro lado, <code>GET</code> , <code>PATCH</code> e <code>DELETE /assignments/{id}</code> existem na API e não são chamados por ninguém. O personal não consegue trocar a ficha de um aluno no meio do ciclo — o uso mais corriqueiro do painel — nem cancelar uma atribuição feita por engano. Só reatribuir do zero, empilhando.
SPA-127	Seletor de exercício some acima de 50 itens	BLOQUEADOR	O builder chama <code>getExercises()</code> sem parâmetro; a API devolve 50 por padrão. O catálogo semeado tem 46 . Sobram quatro de folga. Passando disso, exercícios somem do seletor sem busca, sem paginação e sem aviso — e o personal conclui que "não existe no sistema".
SPA-125	Token de 30 dias em localStorage no painel	ALTA	Seção 8.3. Sem CSP, sem cookie <code>httpOnly</code> .
SPA-129	Bio do personal: 2000 na escrita, 500 na leitura	MÉDIA	Não quebra hoje porque o formulário também trava em 500. Mas qualquer bio entre 501 e 2000 gravada por outro caminho faz o <code>parse()</code> de <code>GET /coaches/me</code> estourar e o painel inteiro parar de carregar . Falha de validação na leitura não degrada, derruba.
SPA-128	Personal criar exercício próprio	MÉDIA	A API suporta exercício próprio por completo, com <code>ownerCoachId</code> e isolamento por tenant. Nenhuma tela usa. O personal fica preso aos 46 do catálogo global, sem lugar para a variação do aparelho da academia dele.
SPA-130	21 vulnerabilidades no ferramental do Expo	BAIXA	Toda a cadeia é ferramenta de build, não código embarcado — risco real baixo. Mas aparece em qualquer auditoria que um cliente peça. A correção sobe o SDK com breaking change; é upgrade de pós-lançamento, não de véspera. API e painel estão com zero vulnerabilidades.

Duas armadilhas embutidas no card mais importante. Quando for religar a persistência de treino (SPA-102), dois descompassos de contrato quebram na primeira chamada: o app gera id de sessão no formato `sess-xxxx` e a API exige UUID; e o índice da série é `min(1)` no app contra `min(0)` na API. Já estão anotados dentro do card.

O que a varredura confirmou estar bem resolvido

- **Zero erro** de `lint` e `typecheck` nos três repositórios. Nenhuma dependência alpha, beta ou rc.
- Estados de carregamento, erro com repetição e lista vazia estão **bem cobertos** no painel — dashboard, alunos, fichas e biblioteca de vídeo, todos tratam os três casos.
- **Cópia em massa de treino** (P0 nº 5 do PRD) e **periodização ABCD** (P1 nº 6) estão implementadas ponta a ponta, com schema idêntico nos três repositórios. Podem sair da lista de pendências.
- A paginação da API está bem desenhada em todos os módulos — o problema está em quem consome, não em quem serve.
- A página pública do personal existe e cobre o essencial. Falta o editor por blocos e os depoimentos que o PRD descreve, mas a versão reduzida é aceitável para o MVP.

11. Conclusão

Três respostas, em ordem de urgência.

Sobre a infraestrutura: a pilha recomendada custa **R\$ 58/mês para lançar** e **R\$ 555/mês com mil alunos**, consumindo menos de 2% da receita bruta nessa escala. A decisão que mais importa não é a de preço — é manter API e banco no mesmo metrô de São Paulo, porque a camada de multi-tenancy faz quatro idas ao banco por requisição e atravessar o hemisfério quatro vezes custa meio segundo por chamada.

Sobre o gratuito: ele existe, é honesto, e economiza R\$ 44 por mês. Não vale a pena — paga-se em cold start, em risco de perda de dado e na sua atenção, que é o recurso escasso aqui.

Sobre o que falta: 35 itens, 16 deles bloqueadores. O volume assusta menos quando se percebe que a maior parte é ligação e operação, não construção. As três coisas para fazer *hoje*, porque destravam todo o resto:

- 1. Abrir as contas Apple e Google.** É o único item com espera de terceiro. Aprovação da Apple leva dias; conta pessoal nova no Google exige 12 testadores por 14 dias antes de liberar produção. Começar tarde atrasa o lançamento mais que qualquer código.
- 2. Registrar o domínio.** É a única decisão irreversível da pilha. Universal link do convite, e-mail que não cai em spam e URL de mídia dependem dele.
- 3. Corrigir o RLS antes de subir qualquer dado real.** Doze tabelas com política ativa e nenhuma com **FORCE**: hoje o isolamento entre pessoais existe só na camada de aplicação, e o banco não tem como socorrer um esquecimento.

Depois disso, a ordem é a da seção 7. E o item de produto que decide se o app cumpre a promessa que faz — fazer o registro de carga chegar ao servidor — é o SPA-102.

Relatório produzido a partir de leitura direta dos três repositórios em 27 de agosto de 2026. Preços verificados nas páginas oficiais dos fornecedores na mesma data; cotação premissa de R\$ 5,40 por dólar. Estimativas estão marcadas como tais no corpo do texto. As pendências estão registradas no Linear, projeto Ironforge, cards SPA-94 a SPA-130.